# Python Imports, Modules and `__init__.py`

A module is a .py file; a package is a folder of modules; `__init__.py` runs when the package is imported.



## Module vs package

| Term | Is | Example |
|---|---|---|
| Module | one `.py` file | `utils.py` → `import utils` |
| Package | folder with `__init__.py` | `shop/` |
| Subpackage | package inside a package | `shop/payments/` |
| Namespace package | folder **without** `__init__.py` (3.3+) | can span several directories |

## Example layout

```
project/
├── main.py
└── shop/                  # package
    ├── __init__.py        # runs on `import shop`
    ├── cart.py            # module
    └── payments/          # subpackage
        ├── __init__.py
        └── card.py
```

## What goes in __init__.py

```
# shop/__init__.py
# re-export -> shop.Cart
from .cart import Cart
from .payments.card import pay

# names `import *` exposes
__all__ = ["Cart", "pay"]
__version__ = "1.0.0"
```

- An empty file is fine – it just marks the folder as a package.
- Runs **once**, on the first import of the package.
- Use it to expose a clean public API, set `__all__` and package metadata.
- Keep it light: heavy work here slows every import.

## Imports inside a package

```
# shop/payments/card.py
from . import refunds            # . = this package
# .. = parent package (shop)
from ..cart import Cart
# absolute: same module
from shop.cart import Cart
```

## Try it: build and import a package

```
import sys, tempfile, pathlib
root = pathlib.Path(tempfile.mkdtemp())
pkg = root / "shop"
pkg.mkdir()
(pkg / "__init__.py").write_text(
    'print("init runs")\nfrom .cart import total\n')
(pkg / "cart.py").write_text(
    "def total(xs):\n    return sum(xs)\n")
sys.path.insert(0, str(root))

import shop              # runs __init__.py once
import shop              # cached: prints nothing
print(shop.total([1, 2, 3]))
```

OUTPUT

```
init runs
6
```

## Four ways to reach the same function

| Statement | Call it as |
|---|---|
| `import shop` | `shop.total(...)` (re-exported in `__init__`) |
| `import shop.cart` | `shop.cart.total(...)` |
| `from shop import total` | `total(...)` |
| `from shop.cart import total as t` | `t(...)` |

## Script or imported? __name__

```
# tool.py
def main():
    print("running")

# True only when run directly
if __name__ == "__main__":
    main()
```

OUTPUT

```
running
```

| Command | `__name__` is |
|---|---|
| `python tool.py` | `"__main__"` |
| `import tool` | `"tool"` |
| `python -m shop.tool` | `"__main__"` (package-aware) |

**PITFALL** Relative imports fail in a file run directly (`ImportError: attempted relative import with no known parent package`). Run it as `python -m shop.module` from the project root.

**TIP** Prefer absolute imports in applications; use relative imports for tightly-coupled siblings inside one package.